

Programação Funcional

Avaliação Preguiçosa

Eliseu C. Miguel

Universidade Federal de Alfenas

17 de junho de 2026

Tópicos

- 1 Introdução
- 2 Conveniência de uso
- 3 Conclusão

Tópicos

- 1 Introdução
- 2 Conveniência de uso
- 3 Conclusão

Tópicos

- 1 Introdução
- 2 Conveniência de uso
- 3 Conclusão

Introdução

A avaliação preguiçosa, *lazy evaluation*, é um conceito fundamental em linguagens de programação funcionais como Haskell. Ela se refere a uma estratégia de avaliação onde a computação de uma expressão é adiada até o momento em que seu valor é realmente necessário para a execução do programa.

Exemplo 1

```
switch :: Int → t → t → t
switch n x y
  | n > 0 = x
  | otherwise = y
```

Nesse caso, se o inteiro n for positivo, a função retornará o valor de x . Caso o contrário, retornará o valor de y . No primeiro caso, o valor de y não é calculado, pois não foi necessário na verificação da expressão e nem no retorno da função. O mesmo ocorre para o segundo caso, o valor de x não é calculado.

Exemplo 2

Definição da função:

```
egFun4 (a,b) = a + 1
```

Aplicando (3+2, 4-17):

```
egFun4 (3+2, 4-17) = (3+2) + 1
```

Nesse caso, o argumento (tupla) é examinado, porém apenas a parte dele utilizada que é calculado, ou seja, apenas o primeiro elemento.

Exemplo 3

Definição da função:

Em Haskell, quando um valor é calculado pela primeira vez em um contexto de avaliação preguiçosa, o valor é armazenado para que não precise ser recalculado. Esse processo é conhecido como *memorization*.

Considere a seguinte função:

```
soma :: Int -> Int
soma x = x + x
calcula :: Int
calcula = soma 5 + soma 5
```

Por mais que "soma 5" seja chamado duas vezes, devido à lazy evaluation, será avaliado uma única vez e seu resultado será reutilizado, tornando o cálculo mais eficiente.

Conveniência

Conveniência de uso

A avaliação preguiçosa se mostra eficiente principalmente na recursão de funções. Os exemplos a seguir, de uma função que verifica se um elemento x existe na lista, possuem computação diferente:

exemplo 1

```
existeNaLista :: Int -> [Int] -> Bool
existeNaLista _ [] = False
existeNaLista x (a:b) = x == a || existeNaLista x b
```

exemplo 2

```
existeNaLista2 :: Int -> [Int] -> Bool
existeNaLista2 _ [] = False
existeNaLista2 x (a:b) = existeNaLista x b || x == a
```

Conveniência

Exemplos

```
existeNaLista :: Int -> [Int] -> Bool
existeNaLista _ [] = False
existeNaLista x (a:b) = x == a || existeNaLista x b
```

```
existeNaLista2 :: Int -> [Int] -> Bool
existeNaLista2 _ [] = False
existeNaLista2 x (a:b) = existeNaLista x b || x == a
```

Por mais que as duas implementações possuem o mesmo objetivo, a computação das duas funções ocorre de maneira diferente. No primeiro caso, se o elemento é encontrado, a recursão da lista para imediatamente. Já no segundo caso, a lista será toda percorrida até começar as comparações para verificar a existência do elemento na lista. Tornando o primeiro caso uma avaliação preguiçosa, por não se interessar em verificar o resto da lista, mas eficiente.

Conclusão

Podemos resumir a avaliação preguiçosa, *lazy evaluation*, em três principais pontos:

- argumentos de funções são calculados apenas quando necessário, ou seja, quando seu valor é necessário para continuar a computação da função para obter o resultado
- um argumento não necessariamente é calculado por completo (como foi mostrado com tuplas)
- um argumento é calculado apenas uma vez

Referências Bibliográficas

Book [1]



Simon Thompson.

Haskell: the Craft of Functional Programming.

Addison-Wesley, Harlow, England, 1997.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos. Ao contrário, é apenas um guia para lhe informar quais são os tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a bibliografia citada no plano de ensino da disciplina para estudar os tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Marcos Vyctor Fonseca Galupo por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.